

Apresentação e Defesa TR1

roteiro completo de 10+ minutos, teoria e excertos de código

Gustavo Henrique Andrade Cavalcanti

Fernando Augusto Hortencio

Teleinformática e Redes 1 - Universidade de Brasília - 2026

Resumo

Este é o material único para apresentação e defesa do Simulador TR1. O roteiro mira 11 a 12 minutos de fala e inclui divisão por integrante, conceitos teóricos, excertos de código e perguntas prováveis. O relatório formal permanece em `relatorio_tr1.pdf`.

1 Planejamento de tempo

Bloco	Tempo	Fala	Foco
Abertura e visão geral	1 min	Fernando	problema, objetivo e fluxo completo
Conceitos antes do código	2 min	Fernando/Gustavo	camadas, enlace, física e ruído
Arquitetura	1 min 30 s	Fernando	threads, filas e meio
Camada de enlace	3 min	Gustavo	quadros, edc, crc e hamming
Camada física e meio	2 min 30 s	Gustavo	sinais, portadora, constelação e ruído
Interface e validação	1 min	Fernando	gtk, testes e resultados
Fechamento	30 s	Fernando	conclusão

2 Ajuste de discurso para a primeira fase

Como todos os grupos receberam a mesma proposta, a apresentação não precisa tratar o enunciado como novidade. A abertura deve ser curta e deve deslocar o foco para as decisões da nossa entrega:

- onde exatamente o ruído entra;
- como o bit vira amostra em volts;
- como o receptor decide o bit ou símbolo recebido;
- por que EDC vem antes do Hamming no transmissor;
- como os testes provam que o pipeline completo funciona.

Frase-guia: a proposta era comum a todos os grupos; o diferencial da nossa entrega está no pipeline completo, manual e testável: texto, bits, quadros, sinal, ruído, demodulação, verificação e texto recuperado.

3 Abertura - Fernando

Fala sugerida: como todos receberam a mesma proposta, a nossa apresentação vai focar menos em repetir o enunciado e mais em mostrar como implementamos cada decisão. A mensagem começa como texto, vira bits, é organizada em quadros, recebe detecção e correção de erros, vira sinal em volts, atravessa um meio com ruído gaussiano e depois é reconstruída no receptor.

`texto -> bits -> enlace -> física -> meio ruidoso -> física -> enlace -> texto`

Pontos obrigatórios: a interface principal é gtk; tx e rx são separados; o ruído entra no meio; crc, hamming, enquadramento e modulações são manuais; a defesa deve insistir em decisões de projeto, não só listar protocolos.

4 Conceitos antes do código

4.1 Visão de camadas - Fernando

Camada	Papel	Arquivo
Aplicação	texto <-> bits	camada_aplicacao.py
Enlace	quadros, edc, hamming	camada_enlace.py
Física	banda-base e portadora	camada_fisica.py
Meio	ruído e potência	meio_comunicacao.py
Simulação	tx, canal e rx	simulador.py

4.2 Enlace - Gustavo

A camada de enlace resolve três problemas: definir fronteiras de quadro, detectar alterações e, quando hamming está ligado, corrigir erro simples. Os protocolos são: contagem, byte stuffing, bit stuffing, paridade, checksum, crc-32 e hamming(8,4).

4.3 Física - Gustavo

A camada física transforma bits em amostras de tensão. Em banda-base, cada bit vira níveis de tensão. Na portadora, grupos de bits escolhem amplitude, frequência, fase ou ponto de constelação. QPSK usa 2 bits por símbolo; 16-QAM usa 4 bits por símbolo, mas é mais sensível ao ruído.

5 Arquitetura - Fernando

```
th_tx = threading.Thread(target=_rotina_tx, args=(config, fila_tx, resultados))
th_rx = threading.Thread(target=_rotina_rx, args=(config, fila_rx, resultados))

th_tx.start()
th_rx.start()

sinal_limpo = fila_tx.get()
sinal_ruidoso = meio_comunicacao.transmitir(
    sinal_limpo, config["ruído_medio"], config["ruído_sigma"])
fila_rx.put(sinal_ruidoso)
```

Explicação: `th_tx` desce a pilha no transmissor; `th_rx` sobe a pilha no receptor; `fila_tx` leva o sinal limpo ao canal; o meio soma ruído; `fila_rx` entrega o sinal ruidoso ao receptor. Isso mostra que tx e rx não se chamam diretamente.

```
for byte in texto.encode("utf-8"):
    for i in range(7, -1, -1):
        bits.append((byte >> i) & 1)
```

Explicação: o texto vira bytes e cada byte é lido do bit mais significativo ao menos significativo. Essa lista de bits é a entrada da camada de enlace.

6 Camada de enlace - Gustavo

6.1 Pipeline

```
for i in range(0, len(bits), tam_bits):
    bloco = bits[i:i + tam_bits]
    bloco = ADICIONAR_EDC[config["deteccao"]](bloco)
    if config["correcao"] == "hamming":
        bloco = codificar_hamming(bloco)
    payloads.append(bloco)
return ENQUADRAR[config["enquadramento"]](payloads)
```

Explicação: o payload é dividido em blocos; o edc é anexado; hamming protege payload + edc; por fim, o quadro é enquadrado. No receptor a ordem é inversa: desenquadrar, corrigir e verificar.

6.2 Enquadramento

```
if byte in (FLAG_BYTE, ESC_BYTE):
    quadro.append(ESC_BYTE)
quadro.append(byte)
```

No byte stuffing, FLAG_BYTE delimita quadro e ESC_BYTE protege dados especiais.

```
if uns == 5:
    trem.append(0)
    uns = 0
```

No bit stuffing, a flag é 01111110. Depois de cinco bits 1 seguidos, o transmissor insere 0 para impedir que dados imitem a flag.

6.3 CRC-32

```
for bit in reversed(byte):
    if (crc ^ bit) & 1:
        crc = (crc >> 1) ^ POLI_CRC32_REFLETIDO
    else:
        crc >>= 1
return crc ^ 0xFFFFFFFF
```

Explicação: o crc é calculado bit a bit. O xor representa soma em $gf(2)$, o polinômio refletido é 0xedb88320, e o vetor 123456789 deve gerar 0xcbf43926. Não há uso de zlib.

6.4 Hamming(8,4)

```
s1 = (bloco[0] + bloco[2] + bloco[4] + bloco[6]) % 2
s2 = (bloco[1] + bloco[2] + bloco[5] + bloco[6]) % 2
s4 = (bloco[3] + bloco[4] + bloco[5] + bloco[6]) % 2
sindrome = s4 * 4 + s2 * 2 + s1
par_geral = sum(bloco) % 2
```

Síndrome	Paridade geral	Resultado
0	par	sem erro
diferente de 0	ímpar	erro simples corrigível
0	ímpar	erro no bit de paridade geral
diferente de 0	par	erro duplo detectado

Explicação: a síndrome aponta a posição do erro simples. A paridade geral impede que dois erros sejam tratados como um erro simples. O custo é dobrar o tamanho, porque 4 bits viram 8 bits.

7 Camada física e meio - Gustavo

7.1 Banda-base

Em banda-base, bits viram níveis de tensão. NRZ-Polar usa +V e -V; Manchester usa transição no meio do bit; Bipolar AMI usa 0 V para zero e alterna a polaridade dos bits 1.

Para o Manchester, a convenção adotada é a de Tanenbaum/G.E. Thomas: o bit 0 faz transição baixo-para-alto e o bit 1 faz transição alto-para-baixo. A demodulação compara a média da primeira metade do bit com a média da segunda metade.

7.2 Portadora e i/q

Ponto importante para defesa: a modulação gera o sinal limpo. O ruído não faz parte da modulação; ele é somado depois, no meio de comunicação, amostra por amostra.

```
def onda(i, q, ciclos):
    N = AMOSTRAS_POR_SIMBOLO
    amostras = []
    for n in range(N):
```

```

    angulo = 2 * math.pi * ciclos * n / N
    amostras.append(i * math.cos(angulo) - q * math.sin(angulo))
return amostras

```

Explicação: cada símbolo é uma sequência de amostras. *i* multiplica o cosseno e *q* multiplica o seno. Escolher (*i*, *q*) é escolher amplitude e fase.

```

i = 2 / N * soma_cos
q = -2 / N * soma_sen
return i, q

```

Explicação: o receptor estima as componentes *i*/*q* por correlação e depois escolhe o ponto válido mais próximo da constelação.

7.3 QPSK e 16-QAM

QPSK transmite 2 bits por símbolo e usa mapeamento gray. 16-QAM transmite 4 bits por símbolo usando níveis -3 , -1 , 1 , 3 em cada eixo. É mais eficiente, mas os pontos ficam mais próximos, então o ruído afeta mais.

7.4 Meio e potência

```

return [amostra + random.gauss(media, sigma) for amostra in sinal]

```

Explicação: cada amostra recebe ruído gaussiano independente. *media* desloca o sinal; *sigma* controla a intensidade do ruído.

```

return sum(amostra * amostra for amostra in sinal) / len(sinal)

```

Explicação: potência média é a média de v^2 , assumindo resistência de 1 ohm.

8 Interface e validação - Fernando

```

resultado = simulador.executar_simulacao(config)
diagnostico = diagnosticar_camadas(
    resultado["tx_bits_aplicacao"], resultado, config)

```

Explicação: a interface não reimplementa protocolos. Ela chama a função real do simulador, calcula diagnóstico por fase e mostra bits, sinais, métricas e relatório de quadros.

```

bits_123 = app.texto_para_bits("123456789")
ok &= testar("crc32 123456789 == 0xCBF43926",
    enlace.calcular_crc32(bits_123) == 0xCBF43926)

```

Explicação: esse teste valida o crc contra um vetor conhecido. A suíte também cobre aplicação, enquadramentos, edc, hamming, modulações e simulação completa.

9 Fechamento - Fernando

O projeto cumpre o enunciado porque separa transmissor e receptor, usa gtk como interface principal, implementa enlace e física manualmente, usa sinal em volts, injeta ruído gaussiano no meio e valida a transmissão com testes. A principal contribuição é tornar visível o caminho completo da informação: texto, bits, quadros, sinal, ruído e texto recuperado.

10 Perguntas prováveis

1. Como vocês modulam com ruído? Nós não modulamos “com” ruído. Primeiro a camada física gera o sinal limpo a partir dos bits. Depois `meio_comunicacao.transmitir` soma `random.gauss(media, sigma)` em cada amostra. O receptor demodula esse sinal ruidoso.

2. Como foi feita a modulação? Em banda-base, cada bit vira um conjunto de amostras em volts: NRZ usa $+V$ e $-V$, Manchester usa transição no meio do bit e Bipolar AMI alterna a polaridade dos bits 1. Na portadora, cada

símbolo vira uma onda $I \cos -Q \sin$; ASK muda amplitude, FSK muda frequência, QPSK usa 2 bits por símbolo e 16-QAM usa 4 bits por símbolo.

3. O ruído entra antes ou depois da modulação? Depois. A ordem é: bits $\rightarrow$ modulação $\rightarrow$ sinal limpo $\rightarrow$ meio ruidoso $\rightarrow$ sinal recebido $\rightarrow$ demodulação.

4. Qual é o jeito certo de colocar o bit de paridade? No código, a paridade é calculada sobre o bloco inteiro e anexada como um byte de EDC: sete zeros mais o bit de paridade. Ela não é colocada no final de cada byte original da mensagem.

5. Por que não usar zlib para crc? Porque o enunciado proíbe bibliotecas prontas para os protocolos. O crc foi implementado bit a bit.

6. Por que usar paridade se CRC-32 é melhor? Porque o trabalho pede comparar mecanismos. A paridade é simples e tem baixo custo, mas só detecta quantidade ímpar de erros. O CRC-32 detecta muito melhor, principalmente erros em rajada, mas insere 32 bits por bloco.

7. Por que hamming vem depois do edc? Para proteger também os bits de verificação anexados ao payload. No receptor, o Hamming tenta corrigir antes de verificar o EDC.

8. Por que hamming(8,4), e não hamming(7,4)? O bit extra de paridade geral permite detectar erro duplo e mantém alinhamento em bytes.

9. Expliquem o Hamming do projeto. Cada 4 bits de dados viram 8 bits: três bits de paridade de Hamming, quatro bits de dados e um bit de paridade geral. No receptor, a síndrome aponta erro simples corrigível; a paridade geral permite diferenciar erro simples de erro duplo.

10. Quais são as vantagens e desvantagens de NRZ-Polar e 16-QAM? NRZ-Polar é simples e robusto para simulação em banda-base, mas pode ter longas sequências sem transição e componente DC. 16-QAM carrega 4 bits por símbolo e é mais eficiente espectralmente, mas os pontos da constelação ficam próximos e a demodulação fica mais sensível ao ruído.

11. Por que 16-QAM é mais sensível a ruído? Porque os pontos da constelação ficam mais próximos. Uma variação menor já pode empurrar o símbolo para a decisão errada.

12. O que significa “não dá para ver o sinal que está chegando” na portadora? Dá para ver o sinal recebido como amostras ruidosas. O que não dá é olhar a onda e ler diretamente os bits como em um desenho ideal. Por isso o receptor estima I e Q por correlação e escolhe o ponto válido mais próximo da constelação.

13. O que acontece se o ruído aumentar muito? O demodulador passa a errar bits. O Hamming pode corrigir erro simples por bloco, mas erros múltiplos podem ser apenas detectados ou até escapar dependendo do EDC.

14. Como funciona o enquadramento com flags? No byte stuffing, o quadro começa e termina com 0x7E; se 0x7E ou 0x7D aparecem nos dados, o transmissor insere 0x7D antes. No bit stuffing, a flag é 01111110; depois de cinco bits 1 seguidos no payload, o transmissor insere um 0.

15. Como vocês testaram? testes.py cobre conversão texto/bits, os três enquadramentos, paridade, checksum, CRC, Hamming, modulações banda-base, modulações por portadora e simulação completa. O CRC também é conferido com o vetor clássico 123456789 $\rightarrow$ 0xCBf43926.

16. A interface web substitui a gtk? Não. A interface principal é gtk; a web é opcional e chama a mesma função.

17. Como provar que o crc está correto? Pelo vetor clássico 123456789, que deve retornar 0xcbf43926.

11 Checklist de domínio

Fernando: executar_simulacao, _rotina_tx, _rotina_rx, filas, interface gtk, diagnóstico e testes.

Gustavo: enquadramento, crc-32, hamming, onda, correlacionar, qpsk, 16-qam, ruído e potência média.